

RAPPORT DE PROJET

RayTracer

G-OOP-400 — C++17

Module	G-OOP-400 — B4 Professional Writings (B-FRE-400)
Projet	RayTracer — Moteur de rendu par lancer de rayons
Promotion	Epitech 2026
Date	30 Mai 2026
Réalisé par	Yan DOSSOU-LOKO
Destinataire	APE (Assistant Pédagogique Expert)

1. Présentation du projet

Le projet RayTracer est un moteur de rendu par lancer de rayons développé en C++17 dans le cadre du module G-OOP-400 à Epitech. L'objectif est de produire des images de synthèse en simulant le trajet de la lumière depuis une caméra virtuelle vers les objets d'une scène 3D, en calculant les intersections rayon/surface et en appliquant un modèle d'éclairage physique.

Le programme se distingue par une architecture modulaire et extensible via un système de plugins dynamiques (.so), ce qui permet d'ajouter de nouvelles primitives ou lumières sans modifier le cœur du moteur. Il supporte deux modes de fonctionnement : un rendu en ligne de commande (CLI) et une interface graphique interactive via SFML.

Le dossier de travail est structuré comme suit :

- **assets** : Contient les images et polices nécessaires au fonctionnement de la partie graphique.
- **include** : Regroupe les fichiers d'en-tête (.hpp) dans des sous-dossiers thématiques (primitives, renderer, camera, lights...) pour la scalabilité et la propreté du code.
- **src** : Miroir de include, contient les fichiers d'implémentation (.cpp) organisés selon les mêmes sous-dossiers.
- **results** : Dossier de sortie où sont stockées les images PPM générées après exécution.
- **plugins** : Contient les bibliothèques dynamiques (.so) des primitives, chargées automatiquement au démarrage du programme.
- **scenes** : Fichiers de configuration (.cfg) décrivant les scènes à rendre.

2. Architecture générale

L'architecture est organisée en couches strictement séparées, chaque couche communiquant avec la suivante exclusivement via des interfaces abstraites. Cette approche garantit un couplage faible et une extensibilité maximale.

2.1 Modules principaux

- **primitives/** : IPrimitive, AbstractPrimitive, Sphere, Plane, Cylinder, Cone, Torus, Triangle, TransformedPrimitive, HitRecord — ensemble des formes géométriques intersectables.
- **math/** : Vec3 (vecteur 3D), Mat4 (matrice 4×4), Ray — fondations mathématiques du moteur.
- **lights/** : ILight, AmbientLight, DirectionalLight, PointLight, Color, computeLighting() — gestion complète de l'éclairage.
- **renderer/** : Renderer (CLI), SfmIOutput (GUI), FrameBuffer, PpmOutput, OutputFactory, LoadFiles — pipeline de rendu et sorties.
- **camera/** : Camera, Rectangle3D, configCamera() — gestion du point de vue et génération des rayons.
- **parser/ & builder/** : SceneParser (libconfig) et SceneBuilder — lecture des fichiers .cfg et construction de la scène runtime.

2.2 Système de plugins dynamiques

Le système de plugins repose sur le chargement dynamique de bibliothèques partagées (.so) via `dlopen/dlsym`. Chaque plugin exporte deux fonctions C : `create()` qui instancie la primitive, et `getType()` qui retourne son identifiant textuel. Le moteur charge automatiquement tous les .so présents dans le dossier `plugins/` au démarrage.

L'utilisation d'extern "C" est indispensable pour éviter le name mangling C++ et permettre à `dlsym` de retrouver les symboles par leur nom exact. La classe `PluginLoader` encapsule entièrement l'API POSIX, et la `PluginFactory<T>` générique gère aussi bien les primitives que les lumières avec une seule implémentation template.

3. Primitives géométriques

Six primitives géométriques principales ont été implémentées, auxquelles s'ajoutent deux primitives avancées (`MobiusStrip`, `TangleCube/Fractal`), chacune héritant de `AbstractPrimitive` qui elle-même implémente `IPrimitive`.

- **Sphere** : Équation quadratique $at^2 + bt + c = 0$. Normale calculée comme (point – centre) / rayon.
- **Plane** : Produit scalaire $t = (C-O) \cdot N / (D \cdot N)$. Normale constante, rejet si le rayon est parallèle au plan.
- **Cylinder** : Projection sur le plan XZ + validation des bornes Y. Hauteur finie, normale horizontale.
- **Cone** : Équation quadratique avec pente $k = r/h$. Apex en haut, hauteur limitée.
- **Torus** : Équation quartique résolue par la méthode de Ferrari. Plus coûteuse en calcul, sélection de la plus petite racine réelle positive.
- **Triangle** : Algorithme de Möller-Trumbore (coordonnées barycentriques). Support du backface culling.
- **MobiusStrip** : Surface paramétrique sans formule analytique d'intersection. Résolution numérique par Newton-Raphson avec calcul des dérivées partielles et du jacobien.
- **TangleCube / Fractal** : Primitives avancées complétant le catalogue, chargées via le système de plugins.

Le décorateur **TransformedPrimitive** enveloppe n'importe quelle primitive avec une matrice de transformation 4×4 (translation, rotation, scale). Le rayon est transformé en espace local avant l'intersection, puis le point et la normale sont ramenés en espace monde via la matrice inverse-transposée, garantissant la correction des normales sous des transformations non-uniformes.

4. Système d'éclairage

Trois types de lumières ont été implémentés, tous héritant de l'interface `ILight`. La fonction `computeLighting()` orchestre leur accumulation pour chaque pixel intersecté.

- **AmbientLight** : Éclairage uniforme sans direction. Calcul : `surfaceColor × intensity`. Indépendant de la géométrie — tous les points reçoivent la même contribution.

- **DirectionalLight** : Source infiniment lointaine (type soleil). Modèle de Lambert : contribution = $\text{surfaceColor} \times \text{diffuse} \times \max(0, \text{normal} \cdot \text{lightDir})$. Pas d'atténuation.
- **PointLight** : Source ponctuelle avec atténuation quadratique : $1 / (1 + 0.0001 \times d^2)$. Combine le modèle de Lambert avec la décroissance physique de la lumière selon la distance.

5. Pipeline de rendu

Le même algorithme est utilisé dans les deux modes (CLI et GUI). Pour chaque pixel (w, h), les étapes suivantes sont exécutées :

- **Lancer de rayon** : Normalisation des coordonnées $u = w/(\text{maxX}-1)$, $v = h/(\text{maxY}-1)$, puis appel à `camera.throwRay(u, v)` qui génère un rayon depuis l'origine caméra vers le point correspondant sur le plan de projection (`Rectangle3D`).
- **Détection d'intersection** : Parcours de toutes les primitives de la scène, conservation de l'intersection la plus proche (t minimal).
- **Calcul de l'éclairage** : Si une intersection existe, appel à `computeLighting()` qui cumule les contributions de toutes les sources lumineuses.
- **Écriture du pixel** : La couleur finale (clampée à $[0,1]$) est convertie en valeurs entières $[0,255]$ et stockée dans le `FrameBuffer`.

Deux modes de rendu

- **Render (CLI)** : Rendu complet en une seule passe. Charge une scène .cfg, calcule tous les pixels et exporte via `PpmOutput` (fichier horodaté). Résolution configurable via `Dimensions`.
- **SfmlOutput (GUI)** : Fenêtre SFML 1600×800 avec panneau de sélection de scènes. Rendu progressif ligne par ligne (une ligne par frame), texture mise à jour en temps réel. Export PPM disponible via bouton. Résolution fixée à 800×800.

6. Design patterns utilisés

- **Factory Pattern** : Création dynamique d'objets sans connaître leur type concret (`PluginFactory<T>`, `PrimitiveFactory`). Un seul template générique gère à la fois les primitives et les lumières. La factory associe une clé textuelle («sphere») à une fonction `create()` résolue une seule fois.
- **Builder Pattern** : Construction progressive d'une scène complexe à partir des données brutes du parser (`SceneBuilder`). Sépare clairement le parsing (production de structures de données) du runtime (création d'objets fonctionnels). Centralise la logique de création.
- **Decorator Pattern** : Ajout de transformations matricielles à n'importe quelle primitive sans la modifier (`TransformedPrimitive`). Composable et transparent pour le render — la primitive décorée répond à la même interface `IPrimitive`.
- **Singleton Pattern** : Instance unique de la factory, thread-safe en C++11+ (`PrimitiveFactory::getInstance()`, Meyers' singleton). Initialisation paresseuse, point d'accès global contrôlé.

- **Template Method** : La classe de base définit la structure de l'algorithme (`AbstractPrimitive::getNormalAtPoint`), les sous-classes implémentent l'étape spécifique (`calculateNormal`). Évite la duplication de code tout en imposant un contrat d'interface clair.
- **RAII** : Gestion automatique de la mémoire via `unique_ptr` et `PluginLoader`. Les plugins sont fermés (`dlclose`) automatiquement à la destruction du loader. Aucune fuite mémoire possible.

7. Difficultés rencontrées et apprentissages

- **Résolution de la surface de Möbius** : La surface de Möbius ne possède pas de formule d'intersection analytique. Il a fallu implémenter la méthode de Newton-Raphson pour approximer numériquement la solution de $\text{ray}(t) = \text{surface}(u,v)$, impliquant le calcul des dérivées partielles ($dPdu$, $dPdv$), du jacobien et une gestion rigoureuse de la convergence.
- **Gestion des plugins dynamiques (dlopen/dlsym)** : L'intégration du système de plugins a nécessité une bonne maîtrise de la liaison dynamique sous Linux. En particulier, la compréhension du name mangling C++ et l'utilisation d'extern "C" ont été des points clés. La gestion de la durée de vie des handles (`dlclose`) via RAII a été un exercice pratique important de gestion de ressources.
- **Transformation des normales** : Les normales de surface ne se transforment pas comme les points géométriques. Sous des transformations non-uniformes (scale différent par axe), l'utilisation de la matrice inverse-transposée est indispensable pour préserver l'orthogonalité des normales par rapport aux surfaces.
- **Architecture modulaire et découplage** : Concevoir une architecture où le moteur ne connaît jamais les types concrets (jamais `Sphere` ou `PointLight` directement, toujours `IPrimitive` et `ILight`) demande une discipline de conception rigoureuse. L'application cohérente des design patterns `Factory`, `Builder` et `Decorator` a permis d'atteindre ce niveau de découplage tout en maintenant une codebase lisible.

8. Bilan et perspectives

Ce projet a permis de mettre en pratique de nombreuses compétences acquises au cours de la formation : programmation orientée objet avancée en C++17, conception architecturale avec des design patterns reconnus, mathématiques appliquées au rendu 3D (algèbre linéaire, géométrie analytique, méthodes numériques), et maîtrise des outils système Linux (bibliothèques dynamiques, `Makefile`).

L'architecture orientée plugins est particulièrement satisfaisante : il est possible d'ajouter une nouvelle primitive géométrique sans recompiler le moteur, simplement en déposant un nouveau fichier `.so` dans le dossier `plugins/`. Cette extensibilité démontre la valeur concrète des abstractions et du découplage en génie logiciel.

Des améliorations sont envisageables : implémentation de l'anti-aliasing par supersampling, ajout d'ombres portées, support de matériaux (réflexion, réfraction), parallélisation du rendu via des threads pour exploiter les architectures multi-cœurs, et exportation dans des formats d'image plus répandus (PNG, JPEG).

